

Module 8: Week 9: Working with React – Part 1

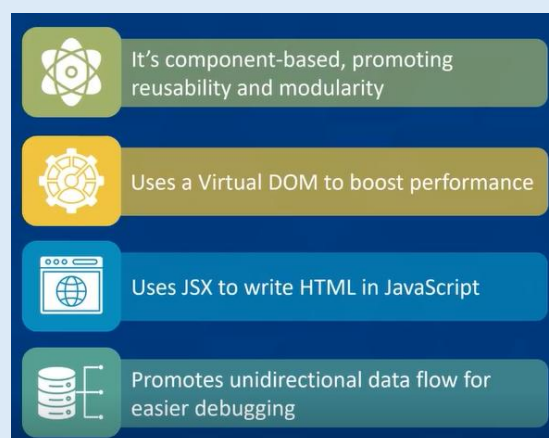
Module Overview

- React is introduced as a widely used JavaScript library that enables developers to build modern, efficient, and scalable single-page applications.
- Core concepts such as props, state, event handling, and component types are explained to help structure dynamic and interactive user interfaces.
- Essential JavaScript ES6 features, including arrow functions, destructuring, and module imports/exports, are integrated as foundational tools for React development.
- Practical guidance is provided on setting up a React environment, handling side effects with hooks like `useEffect` and `useRef`, and working with APIs.
- Navigation and routing using React Router, along with project setup using Create React App, are covered to support scalable application development.



Introduction to React

- React is a JavaScript library developed by Facebook that simplifies UI development using reusable, self-contained components and a virtual DOM.
- React offers advantages such as improved performance, component reusability, easier debugging through unidirectional data flow, and strong community support.



- JSX enables writing HTML-like syntax within JavaScript, improving readability and structure, but requires transpilation via Babel to work in browsers.

- Setting up React involves linking React, ReactDOM, and Babel scripts, and configuring the HTML structure with a root div and JSX-enabled script.

```

<!-- Get the CDN links from the React installation page -->
<script crossorigin src="https://unpkg.com/react@18/umd/react.development.js"></script>
<script crossorigin src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>

<div id="root"></div>

const root = ReactDOM.createRoot(document.querySelector("#root"));
root.render(React.createElement("p", null, "Hello World"));

```

- Components in React can be created using class or function syntax, with function components being simpler, more modern, and preferred for new development.

```

class MenuComponent extends React.Component {
  render() {
    return (
      <div>
        <a href="http://www.google.com">Google</a><br />
        <a href="http://www.yahoo.com">Yahoo</a><br />
        <a href="http://www.facebook.com">Facebook</a><br />
      </div>
    );
  }
}

//can also write this as a function instead of a function value
const MenuComponent = function () {
  return (
    <div>
      <a href="http://www.google.com">Google</a><br />
      <a href="http://www.yahoo.com">Yahoo</a><br />
      <a href="http://www.facebook.com">Facebook</a><br />
    </div>
  );
};

```

Working with React Props

- Props, short for properties, are used in React to pass data to components, enabling reusability and dynamic behaviour.



Enables reusability by allowing behaviour customisation with different data values



Provides a way to pass values to components

```
<GreetingComponent first="John" last="Doe" />
```

- JavaScript expressions can be evaluated within JSX by enclosing them in curly braces, including numbers, objects, and variables.

```
function TestComponent() {
  const n = 10;
  const str = "blar blar blar";
  const person = { name: "John", age: 10 };
  return (
    <div>
      <p>n = {n}</p>
      <p>str = {str}</p>
      <p>
        Name: {person.name}, Age: {person.age}
      </p>
    </div>
  );
}
```

Use { } to pass values as props when they are not string type

```
<MyComponent field1="some_string" field2={10} field3=["item1", "item2"]
field4={{ name: "John", age: 10 }} />
```

```
//props of MyComponent
{
  "field1": "some_string",
  "field2": 10,
  "field3": ["item1", "item2"],
  "field4": { "name": "John", "age": 10 }
}
```

- Arrays can be rendered using the map() function, and each item must have a unique key to optimise React's rendering process.

Can use the map function (or any JS function)

```
function TestArrayComponent(props) {
  //list case: each element should have a unique "key"
  const data = props.data.map(function (item, index){
    return <li key={index}>{item}</li>;
  });

  //make sure to enclose everything using a ul (single root!)
  return <ul>{data}</ul>;
}
```

```
<TestArrayComponent data={["item1", 2, "item3"]} />
```

- Multiple JSX elements must be wrapped in a single root element, often achieved using React fragments to avoid unnecessary nesting.

React fragments enable returning multiple elements without introducing an extra DOM node, ensuring JSX has a single root element.

```
function TestArrayComponent(props) {  
  function generateSomeUI1() {  
    //make sure to enclose everything using a single root  
    return (  
      <div>line 1</div><div>line 2</div>  
    </div>  
    );  
  }  
  function generateSomeUI2() {...}  
  return (<div> {generateSomeUI1()} {generateSomeUI2()} </div>);  
}
```

- The props.children property allows components to receive nested content between opening and closing tags, supporting more flexible and customisable component structures.

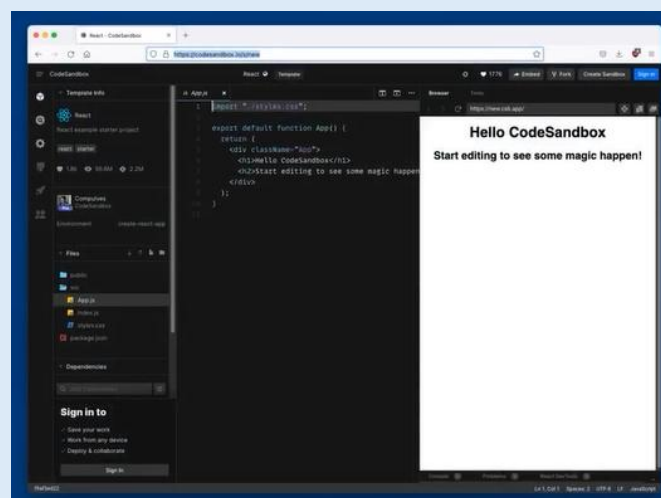
ES6 Module System and Organising React Applications

- React applications are structured using the ES6 module system, where each component is stored in its own file and treated as a module.
- Components are typically exported using export default, and imported using consistent, descriptive names to maintain clarity and prevent confusion.

```
function GreetingComponent(props) {  
  return <div>Hello {props.name}!</div>;  
}  
  
export default GreetingComponent;
```

```
//Alternatively  
export default function GreetingComponent(props) {  
  return <div>Hello {props.name}!</div>;  
}
```

- CodeSandbox offers a cloud-based development environment that eliminates local setup, allowing immediate access to a pre-configured React workspace.



- A typical React project structure includes index.js as the entry point and App.js as the main component that renders the overall application.
- Separating components into individual files improves code readability, reusability, and scalability, supporting better organisation in large-scale React projects.

Modern ES6 Constructs

- Arrow functions provide a more concise syntax for writing functions and are frequently used in modern JavaScript and React development.

```
const add = function (n1, n2) {
  return n1 + n2;
};
```

```
//arrow function (equivalent)
//return is optional (assumed)
const add = (n1, n2) => n1 + n2;
```

```
const items = ["a", "b", "c", "d"];
```

```
[
  "<li index='0'>a</li>",
  "<li index='1'>b</li>",
  "<li index='2'>c</li>",
  "<li index='3'>d</li>",
];
```

- Template strings support multi-line formatting and allow string interpolation using backticks and \${} syntax, improving code clarity and flexibility.

Allows the use of backticks (`) to create multi-line strings and to support string interpolation

```
const person = {
  name: "John",
  location: "Singapore",
};

const greetings = `Welcome ${person.name}!
I know you are from ${person.location}.`;

console.log(greetings);
//Welcome John!
//I know you are from Singapore.
```

String interpolation also works with expressions using \${ }

```
function formatMessage(gender, firstName, lastName) {
  return `Hi ${gender === "M" ? "Mr" : "Ms"} ${firstName} ${lastName}`;
}

const msg1 = formatMessage("M", "John", "Doe");
console.log(msg1); //Hi Mr John Doe

const msg2 = formatMessage("F", "Jane", "Doe");
console.log(msg2); //Hi Ms Jane Doe
```

- Object literal shorthand enables simpler object declarations when variable names match property names, reducing redundancy and improving readability.

```
//previously
const name = "John"; const age = 20;
const person = {
  name: name,
  age: age,
};

//now
const name = "John"; const age = 20;
const person = {
  name,
  age,
};
```

- Rest and spread operators simplify data manipulation by grouping or expanding elements in arrays, objects, and function arguments using '...' syntax.

```
//...items group up the "3rd and later" function call parameters
function generateGreeting(name, gender, ...items) {
  let title = gender === "M" ? "Hi! Mr" : "Hi! Ms";

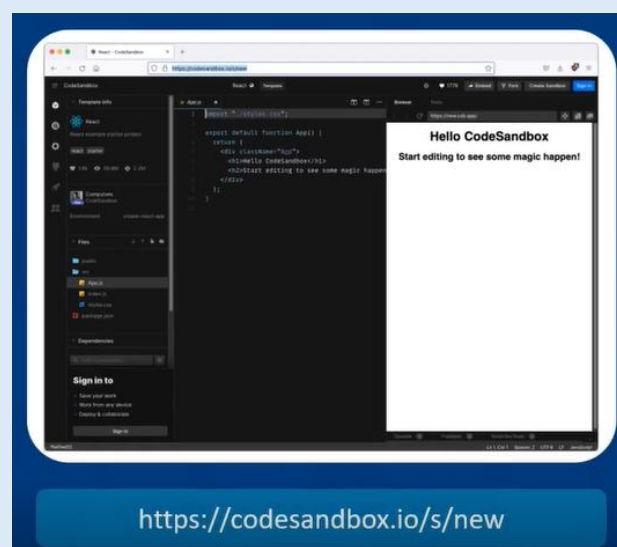
  if (items.length > 0) {
    return `${title} ${name} has ${items}`;
  } else {
    return `${title} ${name}`;
  }
}

console.log(generateGreeting("John", "M", "iPad", "iPhone"));
//Hi! Mr John has iPad,iPhone
```

- Destructuring allows for direct extraction of values from arrays or objects, enabling cleaner code and easier access to required properties or elements.

Hands on Exercise Building a Profile Viewer

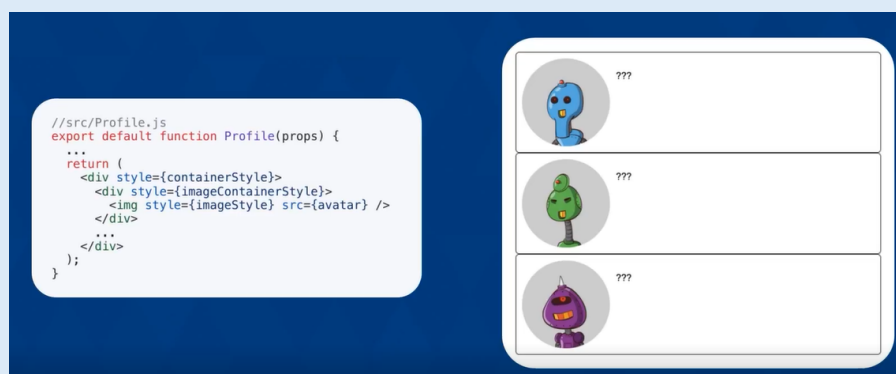
- CodeSandbox provides a quick setup for building a React profile viewer using hardcoded user data and modular components.



- Profile components are created in separate files and styled using JavaScript objects to maintain structure and visual consistency.



- JSX syntax enables dynamic rendering of user profiles by mapping over data arrays and passing props to reusable components.
- Destructuring props simplifies access to user fields like avatar, name and employment details, enhancing readability and maintainability.



- Real-time preview and step-by-step coding reinforce understanding of React fundamentals including component design, styling and data handling.

THE END